Project Overview

- **Project Name**: Maadyaam
- **Project Description**: A high-performance, distraction-free blogging platform allowing users to write, publish, and engage with long-form content. Madhyam provides a seamless reading and writing experience. Users can build personalized feeds by following their favorite authors, interact with articles through likes and bookmarks

2. Requirements

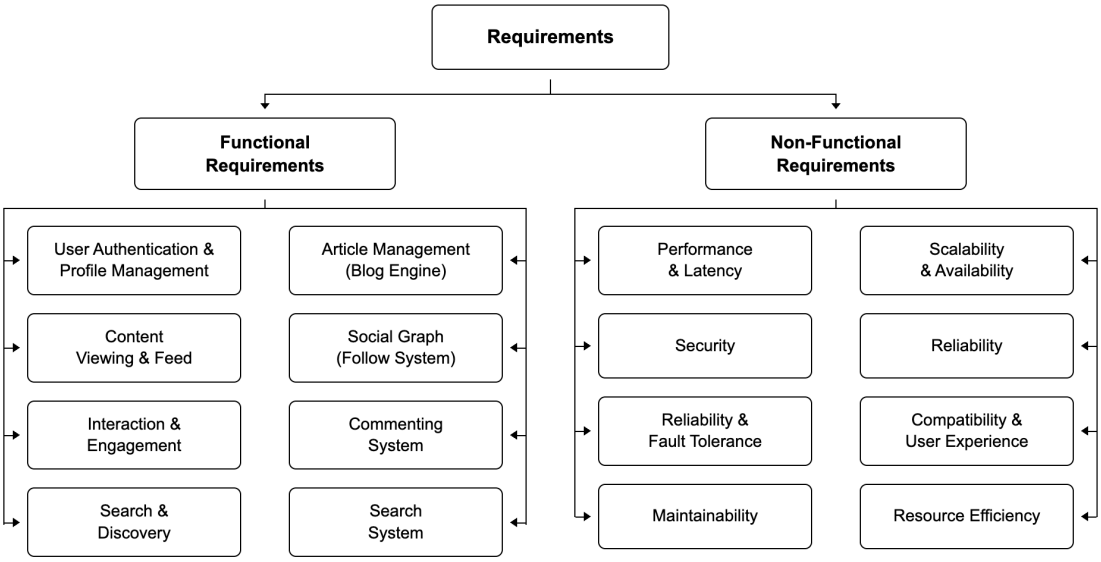


Figure 1 Functional and Non Functional Requirement

Functional Requirements

- **User Authentication & Profile Management:**
  The system shall enable users to register, log in, log out, securely access protected routes, and manage their profile details including updating information, changing passwords, and account deletion.

- **Article Management (Blog Engine):**
  The system shall allow users to create, edit, delete, and publish articles with support for draft and published states, rich-text content, tags, and SEO-friendly slugs.

- **Content Viewing & Feed:**
  The system shall provide users with the ability to view individual articles and access a dynamic feed, including both global and personalized content.

- **Social Graph (Follow System):**
  The system shall enable users to follow and unfollow other users and view followers and following lists.

- **Interaction & Engagement:**
  The system shall allow users to interact with content through claps (likes), sharing, and engagement tracking.

- **Commenting System:**
  The system shall support adding, updating, and deleting comments, including nested replies for threaded discussions.

- **Search & Discovery:**
  The system shall provide search functionality for discovering articles and users based on keywords, tags, and titles.

- **Bookmarking System:**
  The system shall allow users to save, manage, and access bookmarked articles

Non-Functional Requirements:

- **Performance & Latency:**
  The system shall ensure low response times, efficient request handling, and minimal latency for both API responses and content delivery.

- **Scalability & Availability:**
  The system shall support horizontal and vertical scaling while maintaining high availability and uptime.

- **Security:**
  The system shall implement secure authentication, authorization, encryption, and protection against common vulnerabilities such as XSS, CSRF, and injection attacks.

- **Reliability & Fault Tolerance:**
  The system shall ensure consistent operation with minimal downtime, including failover strategies and disaster recovery mechanisms.

- **Maintainability:**
  The system shall follow clean coding standards, modular architecture, and proper dependency management to support easy maintenance and future enhancements.

- **Compatibility & User Experience:**
  The system shall be compatible across devices and browsers while providing a responsive, intuitive, and seamless user experience.

- **Resource Efficiency:**
  The system shall optimize the use of computational resources such as CPU, memory, and storage.

3. System Architecture

- **High-Level Diagram**: The system follows a microservices architecture, where each core functionality is deployed as an independent service inside its own Docker container. These services communicate via REST APIs and are orchestrated for scalability and fault isolation

- **Technology Stack**:
  - **Frontend:** React.js / Next.js (for SEO and server-side rendering), Tailwind CSS
  - **Backend:** Node.js with Express.js (microservices-based architecture)
  - **Database:** MongoDB (MongoDB Atlas)
  - **Infrastructure:** Docker (containerized services), Nginx (API Gateway / reverse proxy), AWS / Render (deployment)
  - **Caching:** Redis (for caching, rate limiting, and token storage)

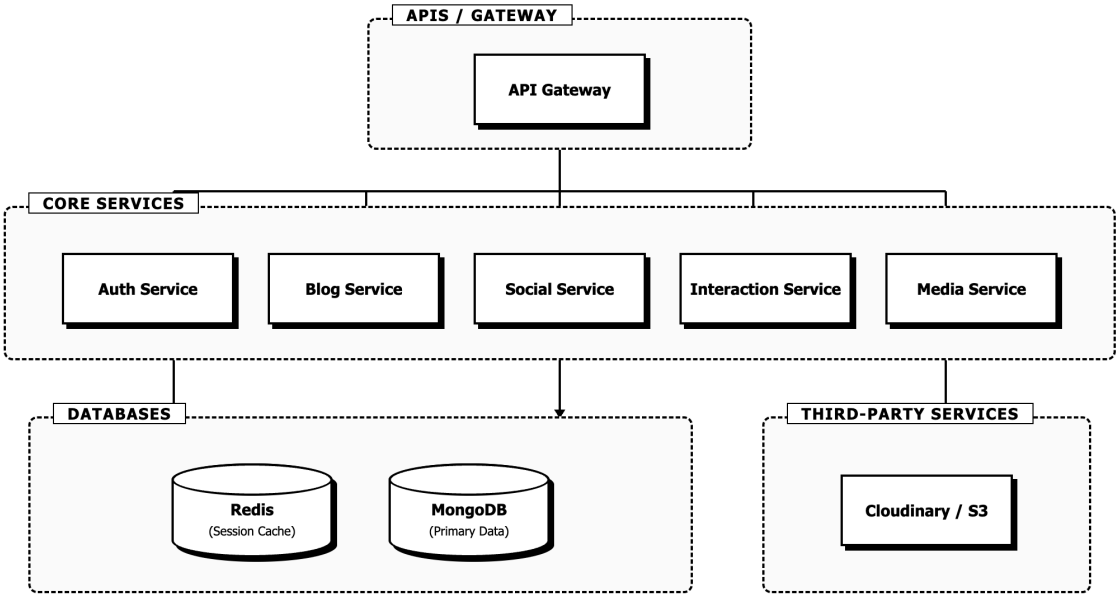


Figure 2 System Component

- **APIs / Gateway:** The single entry point that routes frontend traffic to the correct service, manages rate limits, and handles SSL.
- **Core Services:** Five independent Node.js services handle the business logic: **Auth** (users/JWTs), **Blog** (articles/drafts), **Social** (followers/graph), **Interaction** (likes/comments), and **Media** (file processing).
- **Databases: Redis** provides high-speed session caching (used exclusively by Auth), while **MongoDB** serves as the primary document store for all platform content and relationships.
- **Third-Party Storage: Cloudinary** permanently stores heavy media assets, interacting solely with the Media Service to keep the core databases light.

4. Module Design

1. Auth Service

- **Purpose:** Manages user identity, registration, secure authentication, profile management, and session handling for the entire platform.

- **Inputs:** User credentials (username, email, password for login/signup), JWT refresh tokens (for session renewal), and profile update payloads (bio, displayName, avatar URL).

- **Outputs:** JWT Access Tokens, JWT Refresh Tokens, sanitized User Profile Data (excluding passwords), and HTTP status codes (200, 201, 401, 403).

- **Dependencies:** MongoDB (persistent user data storage), Redis (temporary storage for refresh tokens and session invalidation).

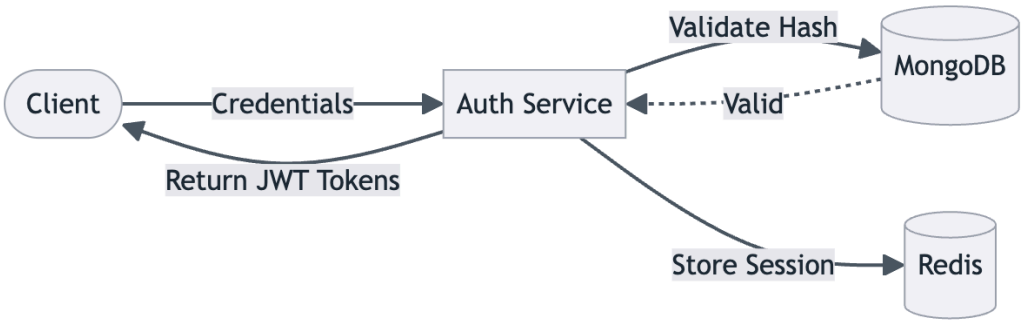


Figure 3 Auth Service Flow (Login)

2. Blog Service

- **Purpose:** Handles the core "Madhyam" content engine, responsible for creating, reading, updating, and deleting (CRUD) blog posts, managing drafts, and handling tags.

- **Inputs:** Post content (title, markdown body, tags), Media IDs (for attached images), User ID (extracted from Auth token), and query parameters (for feed pagination/search).

- **Outputs:** Formatted Blog Post documents, paginated lists of posts (the Feed), search results, and read-time calculations.

- **Dependencies:** MongoDB (Post storage), Auth Service (for route protection and user identity verification).

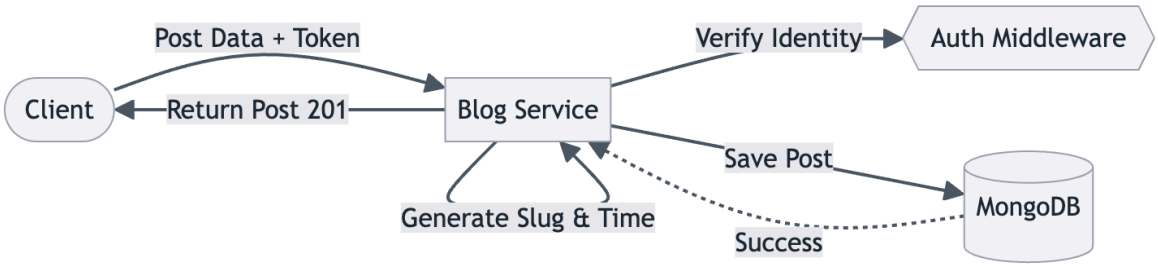


Figure 4 Blog Service Flow

3. Social Service

- **Purpose:** Manages the social graph of the platform, specifically handling the relationships between users (following/unfollowing) to keep the Auth database lightweight.

- **Inputs:** Action Type (Follow/Unfollow), Actor User ID (the person performing the action), and Target User ID (the person being followed).

- **Outputs:** Confirmation of relationship change, updated follower/following counts, and lists of users (e.g., "Get my followers").

- **Dependencies:** MongoDB (Relationship mapping storage), Auth Service (Identity verification)

Figure 5 Social Service Flow

4. Interaction Service

- **Purpose:** Tracks and manages high-volume user engagement data independently, such as likes, unlike, and bookmarking posts for later reading.

- **Inputs:** Post ID, User ID, and Interaction Type (Like/Unlike/Bookmark).

- **Outputs:** Boolean status of interaction (e.g., isLiked: true), total aggregated counts (e.g., 45 Likes), and lists of bookmarked posts for a specific user.

- **Dependencies:** MongoDB (Interaction logs), Auth Service (Identity verification), Blog Service (To verify the target Post actually exists).

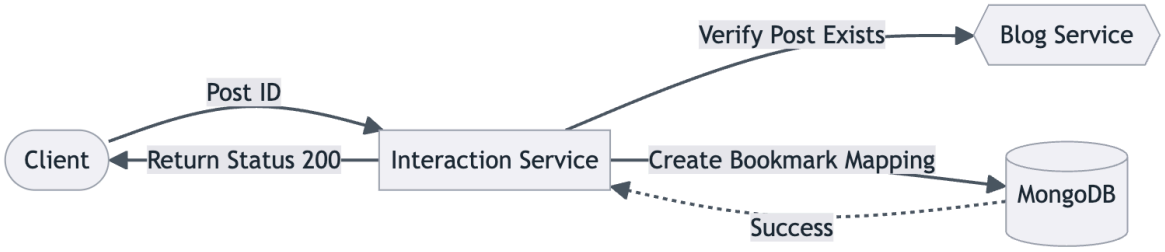


Figure 6 Interaction Service Flow (Bookmark)

5. Media Service

- **Purpose:** Acts as the heavy-lifter for the platform, securely accepting, optimizing, and uploading multimedia files (images, cover photos) to cloud storage, returning accessible URLs..

- **Inputs:** Raw multimedia files (multipart/form-data), file metadata, and the Uploader's User ID.

- **Outputs:** Hosted public Media URLs, unique Media IDs, and file metadata (size, format).

- **Dependencies:** Cloud Storage Provider (Cloudinary), Auth Service (to prevent unauthenticated uploads), Local File System (temporary buffer for processing)

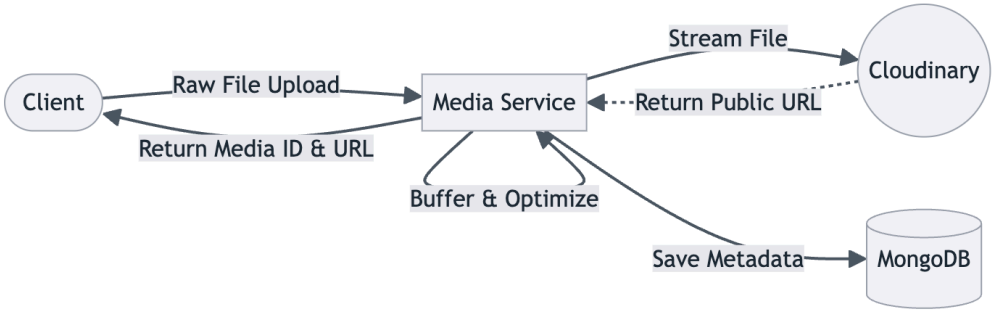
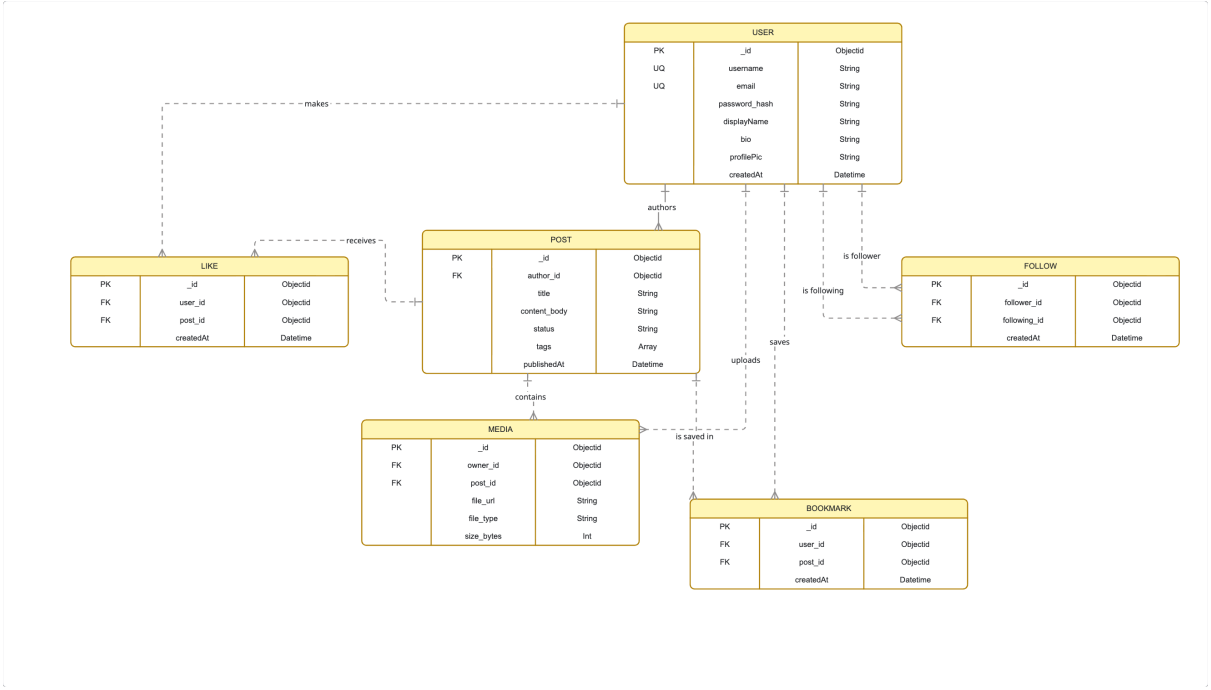


Figure 7 Media Service Flow (Upload Image)

6. Database Design



7. API Design

1. Auth Service (/api/auth/*)

- POST /api/auth/users — Register a new user.
- POST /api/auth/login — Authenticate and receive HttpOnly cookies (Access & Refresh).
- POST /api/auth/token — Refresh the access token using the refresh cookie (validated against Redis).
- DELETE /api/auth/logout — Invalidate the session in Redis and clear auth cookies.
- GET /api/auth/me — Get the currently logged-in user's sanitized profile.
- PATCH /api/auth/profile — Update the current user's profile details (bio, avatar).

2. Blog Service (/api/blog/*)

- GET /api/blog/posts — Get paginated global or personalized feed.
- POST /api/blog/posts — Create a new article (Draft/Published).
- GET /api/blog/posts/:slug — Retrieve a specific article.
- PUT /api/blog/posts/:slug — Update an article (Author/Editor only).
- DELETE /api/blog/posts/:slug — Delete an article (Author/Admin only).

3. Social Service (/api/social/*)

- POST /api/social/follow/:targetUserId — Follow a user.
- DELETE /api/social/unfollow/:targetUserId — Unfollow a user.

- GET /api/social/followers/:userId — List followers for a specific user.
- GET /api/social/following/:userId — List who a specific user is following.

4. Interaction Service (/api/interaction/*)

- POST /api/interaction/like/:postId — Add a like to an article.
- DELETE /api/interaction/like/:postId — Remove a like.
- POST /api/interaction/bookmark/:postId — Save an article for later.
- GET /api/interaction/comments/:postId — List comments for a post.
- POST /api/interaction/comments/:postId — Add a comment to a post.

5. Media Service (/api/media/*)

- POST /api/media/upload — Upload an image (Multipart/form-data) to cloud storage and return the URL.

8. Security Design

- **Authentication/Authorization**:)
  - **JWT via Gateway:** Authentication is stateless for the microservices. Tokens are issued by the auth-service and set as HttpOnly, SameSite=Strict cookies.
  - The frontend never touches the tokens directly in JavaScript. The browser attaches the cookies automatically to requests sent to the Nginx gateway, which can forward the decoded user context to the underlying microservices.

- **Authorization & RBAC**
  - **Role-Based Access Control (RBAC):** The User model defines role: ['user', 'admin', 'editor'].
    - *User:* Can create articles, comment, like, and edit/delete their own resources.
    - *Editor:* Can curate features or hide spam articles across the platform.
    - *Admin:* Full unrestricted access.
  - **Resource Ownership:** Endpoints like PUT /api/blog/posts/:slug verify that req.user.id matches the article's author_id before proceeding.

  - **Rate Limiting**
    - **Infrastructure Limiting:** Nginx is configured to rate-limit the API to 20 requests/second per IP address to prevent brute-forcing and basic DDoS.
    - **Business Logic Quotas (Application Layer):**
      - Maximum 10 articles published per user per day (spam prevention).
      - Maximum 50 likes per user per article (Medium's exact rule).

9. Testing Strategy

- **Unit Testing**
  - **Tools: Jest (Backend) and Jest + React Testing Library (Frontend).**
  - **Focus: Pure functions and isolated components. Testing token generation, password hashing, and ensuring React UI components render the correct inputs and error states based on props.**
- **Integration Testing**
  - **Tools: Supertest + Jest.**
  - **Focus: Module interactions, specifically API routes + Controllers + Database + Redis.**
  - ***Implementation:* Spools up the Express app, connects to an in-memory test MongoDB and Redis instance, executes real HTTP requests (e.g., POST /login), and asserts the Database state and Set-Cookie headers.**

10. Future Enhancements

- **AI-Powered Creator Tools:** Integrating OpenAI or Google Gemini APIs within the Blog Service to offer features like auto-generating post summaries, suggesting SEO-optimized tags, and providing grammar/tone assistance while drafting.

- **Advanced Creator Analytics:** Building an analytics dashboard for authors to track read times, daily views, interaction drop-offs, and audience demographics.
- **Mobile Application:** Developing a cross-platform mobile app (using React Native or Flutter) that consumes the existing microservices via the API Gateway.
- **Algorithmic Feed Generation:** Evolving the feed from a simple chronological list to a machine-learning-driven feed that ranks posts based on user engagement history and tag preferences.